

MANUAL DE UNITY Y GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en macOS con Apple Silicon (M1/M2/M3/M4), Zsh, Keychain, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en macOS (Apple Silicon M1/M2/M3/M4)

Plataforma: macOS Sonoma / Sequoia / versiones modernas con arquitectura ARM64 (Apple Silicon)

Motor: Unity 2022 LTS / Unity 6 (6000.x) para Apple Silicon

Shell: Zsh (Z Shell por defecto de macOS)

Herramientas: Git 2.40+, Git LFS, GitHub CLI (`gh`), Apple Keychain, Homebrew en `/opt/homebrew`, UnityYAMLMerge

Índice de Contenidos

1. [Parte I: Fundamentos y Preparación del Entorno Unity en Apple Silicon](#)
 - 1.1 [Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora](#)
 - 1.2 [Instalación de Git, Git LFS y GitHub CLI con Homebrew ARM64](#)
 - 1.3 [Configuración del Editor de Unity: Visible Meta Files y Force Text](#)
 - 1.4 [La Regla de Oro de los Archivos `.meta` y los GUIDs](#)
 - 1.5 [El Archivo `.gitignore` Oficial y Optimizado para macOS y Unity](#)
 - 1.6 [Configuración Exhaustiva de Git LFS con `.gitattributes`](#)
2. [Parte II: Flujo de Trabajo Esencial Diario \(Nivel Novato\)](#)
 - 2.1 [Inicializar y Publicar un Proyecto de Unity desde Terminal Zsh](#)
 - 2.2 [Clonación Correcta de Proyectos con Git LFS en macOS](#)
 - 2.3 [El Ciclo de Trabajo Seguro: Modificar, Probar y Commitear](#)
 - 2.4 [Conventional Commits Aplicados al Desarrollo de Videojuegos](#)
 - 2.5 [Sincronización sin Romper la Cache \(`pull --rebase` \)](#)
3. [Parte III: Estrategias Colaborativas en Unity \(Nivel Intermedio\)](#)
 - 3.1 [Estrategia de Ramas en Equipos de Videojuegos](#)
 - 3.2 [Arquitectura de Escenas Divididas \(Multi-Scene Editing Aditivo\)](#)
 - 3.3 [Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes](#)
 - 3.4 [Configuración de UnityYAMLMerge en macOS como Mergetool](#)
 - 3.5 [Uso de Git Stash y Worktrees sin Desestabilizar Unity](#)
4. [Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente](#)
 - 4.1 [Tema 1: Prevención Arquitectónica de Conflictos en Unity](#)
 - 4.2 [Tema 2: Conflictos en Archivos `.meta` \(GUID Desincronizado\)](#)
 - 4.3 [Tema 3: Conflictos en Scripts C# \(`.cs` \)](#)

- 4.4 [Tema 4: Conflictos en Escenas \(`.unity` \) y Prefabs con UnityYAMLMerge](#)
 - 4.5 [Tema 5: Forzar una Versión Completa de Asset \(`--ours` vs `--theirs` \).](#)
 - 4.6 [Tema 6: Cambios Locales en el Editor al Hacer Pull](#)
 - 4.7 [Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS](#)
 - 4.8 [Tema 8: Conflictos en Archivos Binarios y Bloqueo con Git LFS Lock](#)
 - 4.9 [Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano](#)
5. [Parte V: Herramientas Modernas de Productividad Avanzada](#)
- 5.1 [Git Worktrees en macOS: Múltiples Ramas sin Recargar Unity](#)
 - 5.2 [Depuración Binaria con Git Bisect y Rastreo con Blame](#)
 - 5.3 [GitHub Codespaces desde la Terminal de Mac](#)
 - 5.4 [GitHub Copilot CLI en Zsh](#)
 - 5.5 [Git Hooks y Validación con Pre-commit en macOS](#)
6. [Parte VI: Gestión de Paquetes UPM y Dependencias](#)
- 6.1 [Instalación de Paquetes de Unity Mediante URLs de Git](#)
 - 6.2 [Creación y Publicación de Paquetes UPM en Repositorios Privados](#)
7. [Parte VII: Automatización CI/CD con GitHub Actions y GameCI](#)
- 7.1 [Arquitectura de GameCI para macOS Standalone \(Apple Silicon\)](#)
 - 7.2 [Activación de Licencias de Unity en GitHub Actions](#)
 - 7.3 [Pipeline Automatizado: Tests y Compilación StandaloneOSX \(.app/.dmg\)](#)
 - 7.4 [Subida Automática de Builds a GitHub Releases](#)
8. [Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos](#)
- 8.1 [Branch Protection Rules y Rulesets](#)
 - 8.2 [Gestión de Secretos para APIs de Juegos \(Apple Game Center, Steam\)](#)
 - 8.3 [Gobernanza con `CODEOWNERS` para Artistas y Programadores](#)
9. [Parte IX: Catálogo Maestro de Incidentes Críticos de Unity en macOS](#)
- 9.1 [Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs](#)
 - 9.2 [Incidente 2: Escena Corrupta por Conflicto Mal Resuelto](#)
 - 9.3 [Incidente 3: Subida Accidental de la Carpeta `Library/` en macOS](#)
 - 9.4 [Incidente 4: Push Rechazado por Archivo Mayor a 100 MB](#)
 - 9.5 [Incidente 5: Shaders Magenta y Metal API tras Conmutar Ramas](#)
 - 9.6 [Incidente 6: Límite de Ancho de Banda de Git LFS Superado](#)
 - 9.7 [Incidente 7: Desfase de Versiones Menores del Editor de Unity](#)
 - 9.8 [Incidente 8: Fuga de Claves en ScriptableObjects o Archivos de Configuración](#)
 - 9.9 [Incidente 9: Problemas con Archivos `.DS\_Store` en Proyectos Unity](#)

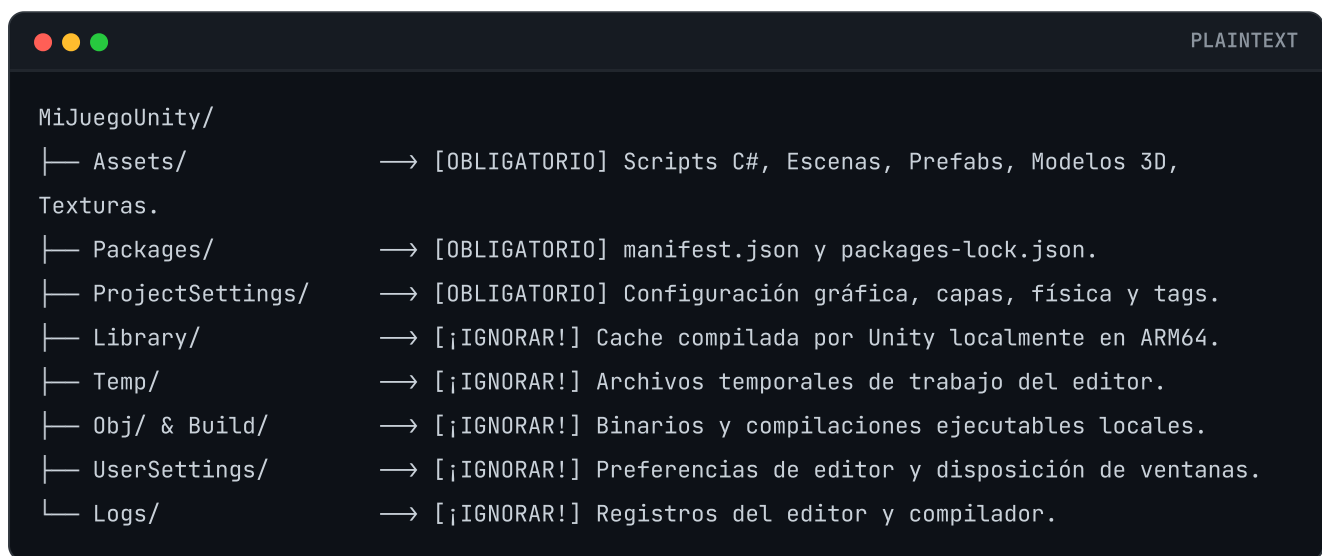
- 9.10 [Incidente 10: Limpieza y Reconstrucción Limpia de la Cache en Apple Silicon](#)
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Apple Silicon

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

Un proyecto de Unity en macOS con procesador Apple Silicon genera una cache de compilación acelerada por hardware dentro de `Library/` que nunca debe rastrearse en Git:



MiJuegoUnity/	
├ Assets/	→ [OBLIGATORIO] Scripts C#, Escenas, Prefabs, Modelos 3D, Texturas.
├ Packages/	→ [OBLIGATORIO] manifest.json y packages-lock.json.
├ ProjectSettings/	→ [OBLIGATORIO] Configuración gráfica, capas, física y tags.
├ Library/	→ [¡IGNORAR!] Cache compilada por Unity localmente en ARM64.
├ Temp/	→ [¡IGNORAR!] Archivos temporales de trabajo del editor.
├ Obj/ & Build/	→ [¡IGNORAR!] Binarios y compilaciones ejecutables locales.
├ UserSettings/	→ [¡IGNORAR!] Preferencias de editor y disposición de ventanas.
└ Logs/	→ [¡IGNORAR!] Registros del editor y compilador.

1.2 Instalación de Git, Git LFS y GitHub CLI con Homebrew ARM64

Abre **Terminal** o **iTerm2** en tu Mac:



```
brew install git git-lfs gh jq
git lfs install
which git
file $(which git)
```

¿Qué hace este comando?

Instala **Git**, **Git LFS** y **GitHub CLI** nativos para procesadores Apple Silicon (ARM64) en `/opt/homebrew/bin`, activa los filtros de LFS y verifica que el binario sea un ejecutable Mach-O de 64 bits ARM64.

1.3 Configuración del Editor de Unity: Visible Meta Files y Force Text

Antes de inicializar Git en cualquier proyecto de Unity:

1. Abre tu proyecto en Unity para Apple Silicon.
2. Ve a **Edit → Project Settings → Version Control**.
 - En **Mode**, selecciona estrictamente: `Visible Meta Files`.
3. Ve a **Edit → Project Settings → Editor**.
 - En **Asset Serialization Mode**, selecciona estrictamente: `Force Text`.

IMPORTANT

`Force Text` obliga a Unity a escribir todas las escenas (`.unity`), prefabs (`.prefab`) y materiales en formato de texto plano YAML. Esto permite realizar comparaciones y fusiones semánticas precisas.

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Cada archivo dentro de `Assets/` posee un archivo `.meta` con un identificador único (GUID).

- Si renombras o mueves un asset en Finder, **debes renombrar o mover su `.meta` idénticamente**.
- Si subes un asset sin su `.meta` a GitHub, el Unity de tus compañeros creará un `.meta` con un GUID diferente, rompiendo los enlaces de escenas y prefabs (*Missing Script* y *Missing Prefab*).

1.5 El Archivo `.gitignore` Oficial y Optimizado para macOS y Unity

Crea `.gitignore` en la raíz del proyecto:

```
/[Ll]ibrary/  
/[Tt]emp/  
/[Oo]bj/  
/[Bb]uild/  
/[Bb]uilds/  
/[Ll]ogs/  
/[Uu]ser[Ss]ettings/  
/[Mm]emoryCaptures/  
  
# Xcode & Rider  
.vs/  
*.csproj  
*.unityproj  
*.sln  
*.suo  
*.user  
*.userprefs  
*.pidb  
*.booproj  
*.pdb  
*.opendb  
  
# Compilaciones  
*.apk  
*.aab  
*.unitypackage  
*.app  
*.dmg  
  
# Metadatos del sistema macOS  
.DS_Store  
.AppleDouble  
.LSOverride  
._*  
.Spotlight-V100  
.Trashes
```

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Crea `.gitattributes` en la raíz del proyecto para fijar finales de línea LF y delegar archivos pesados a Git LFS:

```
# Finales de línea Unix
* text=auto eol=lf
*.cs text diff=csharp
*.shader text
*.hlsl text
*.compute text
*.json text
*.yaml text

# Integración con UnityYAMLMerge
*.unity merge=unityyamlmerge eol=lf
*.prefab merge=unityyamlmerge eol=lf
*.asset merge=unityyamlmerge eol=lf
*.mat merge=unityyamlmerge eol=lf

# Modelos 3D en Git LFS
*.fbx filter=lfs diff=lfs merge=lfs -text
*.obj filter=lfs diff=lfs merge=lfs -text
*.blend filter=lfs diff=lfs merge=lfs -text
*.dae filter=lfs diff=lfs merge=lfs -text

# Texturas y Gráficos en Git LFS
*.png filter=lfs diff=lfs merge=lfs -text
*.jpg filter=lfs diff=lfs merge=lfs -text
*.tga filter=lfs diff=lfs merge=lfs -text
*.psd filter=lfs diff=lfs merge=lfs -text
*.exr filter=lfs diff=lfs merge=lfs -text
*.hdr filter=lfs diff=lfs merge=lfs -text

# Audio y Video en Git LFS
*.wav filter=lfs diff=lfs merge=lfs -text
*.mp3 filter=lfs diff=lfs merge=lfs -text
*.ogg filter=lfs diff=lfs merge=lfs -text
*.mp4 filter=lfs diff=lfs merge=lfs -text

# Binarios y librerías
*.bundle filter=lfs diff=lfs merge=lfs -text
*.dylib filter=lfs diff=lfs merge=lfs -text
*.dll filter=lfs diff=lfs merge=lfs -text
```

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto de Unity desde Terminal Zsh

```
cd ~/Developer/MiJuegoUnity
git init
git lfs install
git add .gitignore .gitattributes
git commit -m "chore: configurar gitignore y gitattributes para Unity LFS"
git add Assets/ Packages/ ProjectSettings/
git commit -m "feat: inicializar estructura principal del videojuego"
gh repo create MiJuegoUnity --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Inicia el repositorio Git, activa Git LFS, commitea los archivos de configuración y sube el proyecto a GitHub con un solo comando.

2.2 Clonación Correcta de Proyectos con Git LFS en macOS

```
git clone git@github.com:usuario/MiJuegoUnity.git
cd MiJuegoUnity
git lfs pull
```

¿Qué hace este comando?

Clona el repositorio y fuerza la descarga de los binarios reales de LFS en tu Mac.

2.3 El Ciclo de Trabajo Seguro: Modificar, Probar y Commitear

1. En Unity, guarda la escena (`Cmd + S`).
2. Espera a que termine la compilación de scripts en Unity.

3. En la terminal:

```
git status -s
git add Assets/Scripts/Heroe.cs Assets/Scripts/Heroe.cs.meta
git commit -m "feat(player): añadir ataque especial cargado"
```

¿Qué hace este comando?

Prepara de forma inseparable el código fuente y su archivo `.meta` con el GUID.

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

```
git commit -m "feat(combat): implementar sistema de parry con ventana de frames"
git commit -m "fix(audio): solucionar retraso de audio en altavoces de Mac"
git commit -m "art(boss): importar modelo 3D y animaciones de Demonio.fbx"
```

2.5 Sincronización sin Romper la Cache (`pull --rebase`)

```
git fetch origin
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Descarga los commits del equipo y sitúa tus commits locales en la cima linealmente. Unity reimportará limpiamente solo los archivos alterados.

Parte III: Estrategias Colaborativas en Unity (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos

```
git switch -c feature/sistema-combate
git push -u origin feature/sistema-combate
```

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

Nunca trabajen todos sobre la misma escena. Dividan cada nivel en sub-escenas:

- `Nivel_01_Core.unity` (GameManagers, cámara principal).
- `Nivel_01_Entorno.unity` (Terreno, iluminación estática).
- `Nivel_01_Enemigos.unity` (Spawners e IA).

Cárguenlas en tiempo de ejecución con:

```
SceneManager.LoadSceneAsync("Nivel_01_Entorno", LoadSceneMode.Additive);
```

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

Encapsula personajes, trampas e interfaces dentro de **Prefabs**. Edita los elementos en el modo de aislamiento de Prefabs para que los cambios se guarden en `.prefab` sin tocar el archivo `.unity` de la escena principal.

3.4 Configuración de UnityYAMLMerge en macOS como Mergetool

Localiza tu instalación de Unity en
`/Applications/Unity/Hub/Editor/<version>/Unity.app/Contents/Tools/UnityYAMLMerge` :

```
UNITY_MERGE="/Applications/Unity/Hub/Editor/6000.0.23f1/Unity.app/Contents/Tools/UnityYAMLMerge"

git config --global merge.unityyamlmerge.name "Unity SmartMerge"
git config --global merge.unityyamlmerge.driver "'$UNITY_MERGE' merge -h -p --mode=落后 %0
%A %B %A"
git config --global merge.unityyamlmerge.recursive binary
```

¿Qué hace este comando?

Configura Git para que invoque el fusionador semántico nativo de Unity en macOS ante cualquier colisión en escenas o prefabs.

3.5 Uso de Git Stash y Worktrees sin Desestabilizar Unity

```
# Extraer una rama de hotfix en una carpeta paralela sin vaciar la cache de Library
git worktree add ../MiJuego-Hotfix hotfix/correccion-urgente
```

¿Qué hace este comando?

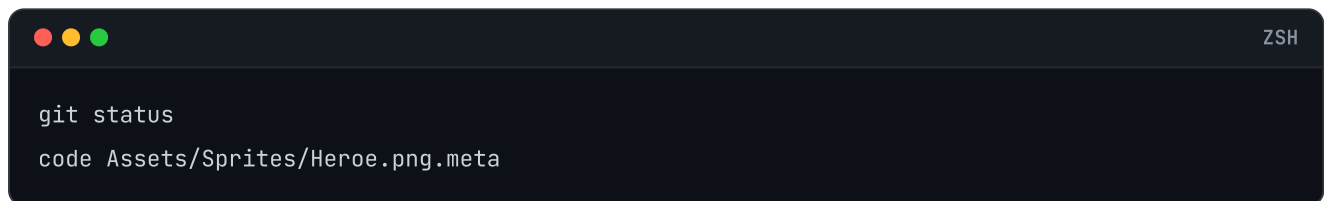
Crea una carpeta física independiente en tu disco. Puedes abrir dos instancias de Unity en macOS simultáneas sin vaciar la cache de compilación de tu proyecto principal.

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

1. **Escenas Aditivas:** Dividir niveles en sub-escenas independientes.
 2. **Uso riguroso de Prefabs:** Toda entidad interactiva es un Prefab.
 3. **Bloqueo exclusivo:** Usar `git lfs lock` para escenas críticas.
-

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)



```
git status
code Assets/Sprites/Heroe.png.meta
```

1. Conserva un único valor en la clave `guid:` . Elige el GUID que ya estaba referenciado en escenas existentes.
2. Elimina los marcadores `<<<<<<` , `=====` y `>>>>>>` .
3. Guarda y ejecuta:



```
git add Assets/Sprites/Heroe.png.meta
git rebase --continue
git push origin main
```

4.3 Tema 3: Conflictos en Scripts C# (.cs)

```
git status
code Assets/Scripts/Heroe.cs
git add Assets/Scripts/Heroe.cs
git rebase --continue
git push origin main
```

4.4 Tema 4: Conflictos en Escenas (.unity) y Prefabs con UnityYAMLMerge

```
git mergetool
```

¿Qué hace este comando?

Ejecuta `UnityYAMLMerge`. Si los `GameObjects` no colisionan en la misma propiedad, Unity reconciliará la escena automáticamente.

Si requiere intervención manual, abrirá VS Code. Revisa, guarda y ejecuta:

```
git add Assets/Scenes/Nivel1.unity
git commit -m "merge: resolver colisión en escena Nivel1"
git push origin main
```

4.5 Tema 5: Forzar una Versión Completa de Asset (--ours vs --theirs)

```
# Conservar mi versión local completa (escena y meta):
git checkout --ours Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git add Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git commit -m "resolve: conservar versión local de la escena"
git push origin main
```

```
# Aceptar la versión remota del compañero completa:
git checkout --theirs Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git add Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git commit -m "resolve: aceptar versión remota del compañero"
git push origin main
```

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

```
git stash push -m "Ajustes locales del inspector"
git pull --rebase origin main
git stash pop
```

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

```
git pull --rebase origin main
git lfs push --all origin main
git push origin main
```

4.8 Tema 8: Conflictos en Archivos Binarios y Bloqueo con Git LFS Lock

```
# Bloquear un modelo 3D antes de empezar a trabajar en él
git lfs lock Assets/Modelos/Heroe.fbx

# Comprobar bloqueos activos en el repositorio
git lfs locks

# Liberar el bloqueo al terminar y subir cambios
git lfs unlock Assets/Modelos/Heroe.fbx
```

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano

```
# Para mantener el asset:
git add Assets/Scripts/Bala.cs Assets/Scripts/Bala.cs.meta
git commit -m "resolve: conservar script y meta"

# Para confirmar el borrado:
git rm Assets/Scripts/Bala.cs Assets/Scripts/Bala.cs.meta
git commit -m "resolve: confirmar borrado de script y meta"
```

ZSH

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en macOS: Múltiples Ramas sin Recargar Unity

```
git worktree add ../MiJuego-Test feature/pruebas-fisica
cd ../MiJuego-Test
git commit -am "test: ajustar inercia de salto en Apple Silicon"
git push origin feature/pruebas-fisica

cd ../MiJuegoUnity
git worktree remove ../MiJuego-Test
```

5.2 Depuración Binaria con Git Bisect y Rastreo con Blame

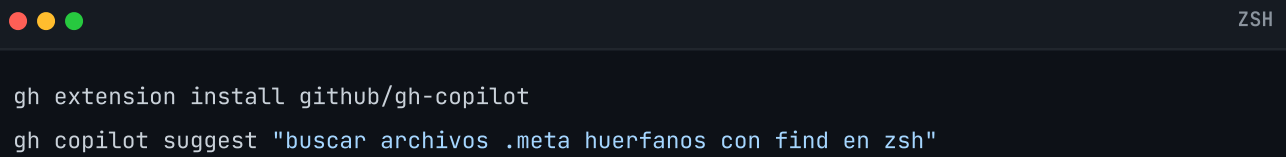
```
git bisect start
git bisect bad
git bisect good v1.0.0

# Probar compilación en Unity y marcar:
git bisect good # 0: git bisect bad
git bisect reset
```

5.3 GitHub Codespaces desde la Terminal de Mac

```
gh codespace create --repo usuario/MiJuegoUnity --branch main
gh codespace code -c nombre-del-codespace
```

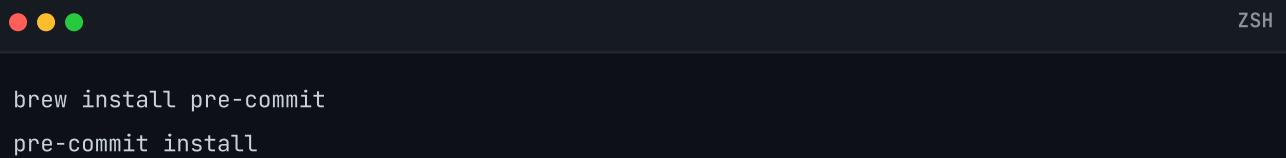
5.4 GitHub Copilot CLI en Zsh



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The terminal contains two lines of text: 'gh extension install github/gh-copilot' and 'gh copilot suggest "buscar archivos .meta huérfanos con find en zsh"'. The text is in a light blue monospaced font.

```
gh extension install github/gh-copilot
gh copilot suggest "buscar archivos .meta huérfanos con find en zsh"
```

5.5 Git Hooks y Validación con Pre-commit en macOS



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The terminal contains two lines of text: 'brew install pre-commit' and 'pre-commit install'. The text is in a light blue monospaced font.

```
brew install pre-commit
pre-commit install
```

Parte VI: Gestión de Paquetes UPM y Dependencias

6.1 Instalación de Paquetes de Unity Mediante URLs de Git

Edita `Packages/manifest.json` :



```
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?
path=src/UniTask/Assets/Plugins/UniTask#2.5.5"
  }
}
```

The screenshot shows a code editor window with a dark theme. The title bar on the right says "JSON". The code is a JSON object with a "dependencies" key containing a single entry for "com.cysharp.unitask" with a URL and a path. The code is formatted with syntax highlighting.

6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados

Estructura de paquetes modulares independientes con su propio `package.json` .

Parte VII: Automatización CI/CD con GitHub Actions y GameCI

7.1 Arquitectura de GameCI para macOS Standalone (Apple Silicon)

GameCI permite ejecutar pruebas y generar ejecutables de macOS empaquetados como `.app` o `.dmg`.

7.2 Activación de Licencias de Unity en GitHub Actions

Configura en los secretos del repositorio: `UNITY_EMAIL`, `UNITY_PASSWORD`, `UNITY_LICENSE`.

7.3 Pipeline Automatizado: Tests y Compilación StandaloneOSX (.app/.dmg)

Crea `.github/workflows/unity-macos-ci.yml`:

```
name: Unity macOS Build CI

on:
  push:
    branches: [ main ]

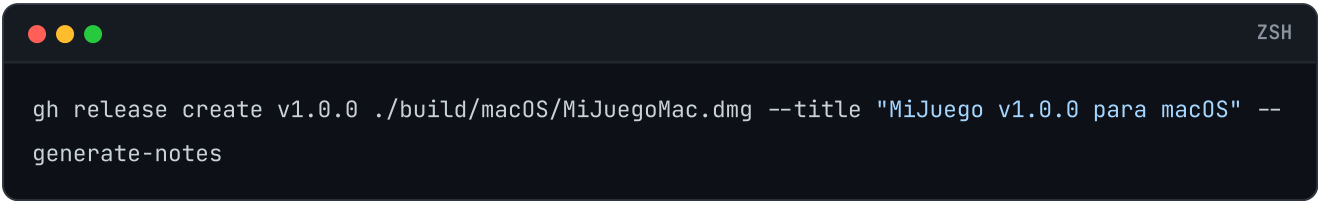
jobs:
  build-macos:
    name: Compilar StandaloneOSX (Apple Silicon)
    runs-on: macos-14 # Runner oficial M1 de GitHub
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v4
        with:
          path: Library
          key: Library-macos-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}

      - uses: game-ci/unity-builder@v4
        env:
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
        with:
          projectPath: .
          targetPlatform: StandaloneOSX
          buildName: MiJuegoMac
          buildsPath: build/macos

      - uses: actions/upload-artifact@v4
        with:
          name: MiJuego-macos-Universal
          path: build/macos/StandaloneOSX
```

7.4 Subida Automática de Builds a GitHub Releases



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "ZSH" is in the top-right corner. The command entered is: `gh release create v1.0.0 ./build/macOS/MiJuegoMac.dmg --title "MiJuego v1.0.0 para macOS" --generate-notes`

```
gh release create v1.0.0 ./build/macOS/MiJuegoMac.dmg --title "MiJuego v1.0.0 para macOS" --generate-notes
```

Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

8.1 Branch Protection Rules y Rulesets

- Prohibir commits directos a `main`.
 - Requerir que los tests automatizados de GameCI terminen en verde.
 - Bloquear force-pushes (`git push --force`).
-

8.2 Gestión de Secretos para APIs de Juegos (Apple Game Center, Steam)

Nunca incluyas certificados `.p12` de Apple Developer o API keys privadas en scripts de C# en el repositorio.

8.3 Gobernanza con CODEOWNERS para Artistas y Programadores

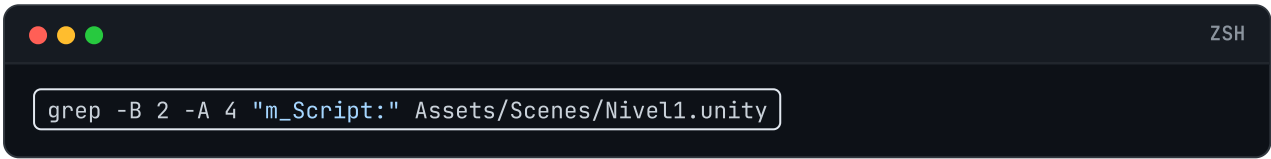
A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text "PLAINTEXT" is visible in the top-right corner. The terminal displays the content of a .github/CODEOWNERS file.

```
# .github/CODEOWNERS
/Assets/Scripts/ @estudio/programadores
/Assets/Modelos/ @estudio/artistas-3d
/ProjectSettings/ @estudio/lead-developer
```

Parte IX: Catálogo Maestro de Incidentes Críticos de Unity en macOS

9.1 Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs

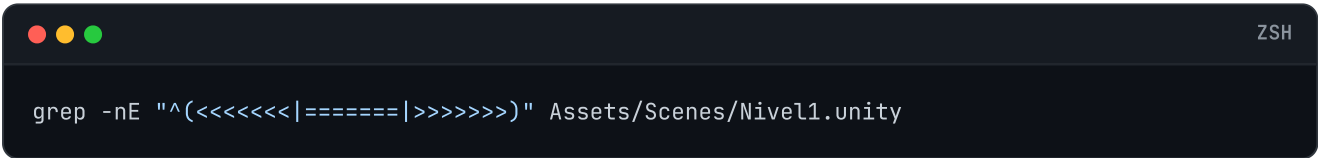
1. Abre la escena en VS Code y localiza el GUID faltante:



```
grep -B 2 -A 4 "m_Script:" Assets/Scenes/Nivel1.unity
```

2. Restaura ese GUID exacto en el archivo `.meta` del script correspondiente.

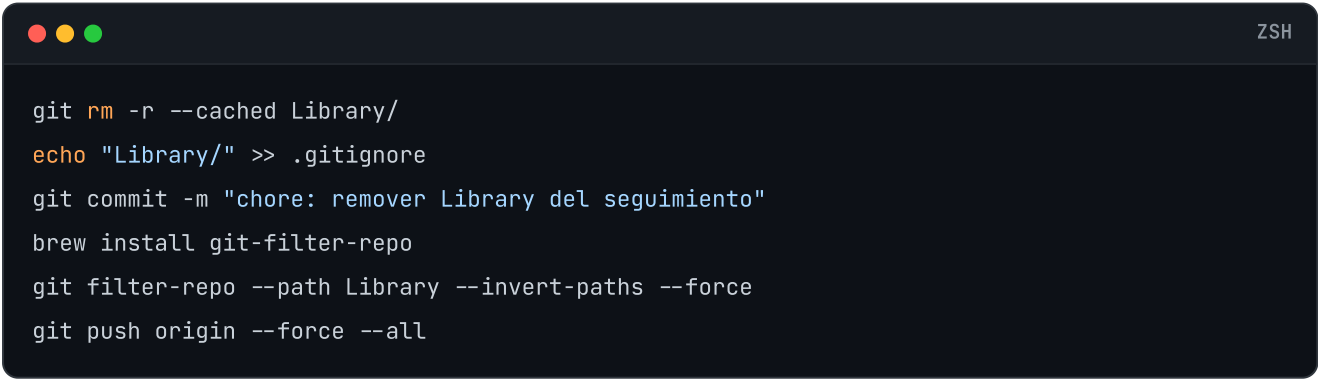
9.2 Incidente 2: Escena Corrupta por Conflicto Mal Resuelto



```
grep -nE "^(<<<<<<|=====>>>>>>)" Assets/Scenes/Nivel1.unity
```

Elimina los marcadores en VS Code y guarda el archivo.

9.3 Incidente 3: Subida Accidental de la Carpeta `Library/` en macOS



```
git rm -r --cached Library/  
echo "Library/" >> .gitignore  
git commit -m "chore: remover Library del seguimiento"  
brew install git-filter-repo  
git filter-repo --path Library --invert-paths --force  
git push origin --force --all
```

9.4 Incidente 4: Push Rechazado por Archivo Mayor a 100 MB

```
git lfs migrate import --include="*.fbx,*.wav,*.mp4,*.psd" --everything
git push origin --force --all
```

9.5 Incidente 5: Shaders Magenta y Metal API tras Conmutar Ramas

En macOS con Apple Silicon, los shaders se compilan para la API Metal. Si aparecen morados o magenta:

1. En Unity, ve a **Edit → Project Settings → Graphics**.
2. Haz clic derecho sobre la carpeta `Assets/Shaders` o `Assets/Materials` y selecciona **Reimport**.

9.6 Incidente 6: Límite de Ancho de Banda de Git LFS Superado

Amplía la cuota de LFS en GitHub o configura un endpoint LFS privado.

9.7 Incidente 7: Desfase de Versiones Menores del Editor de Unity

Comprueba `ProjectSettings/ProjectVersion.txt` para asegurar que todo el equipo utilice exactamente la misma versión del motor.

9.8 Incidente 8: Fuga de Claves en ScriptableObjects o Archivos de Configuración

```
git filter-repo --path Assets/Resources/ConfiguracionSecreta.asset --invert-paths --force
git push origin --force --all
```

9.9 Incidente 9: Problemas con Archivos .DS_Store en Proyectos Unity

```
find . -name .DS_Store -print0 | xargs -0 git rm --ignore-unmatch --cached  
git commit -m "chore: purgar archivos .DS_Store del proyecto Unity"
```

9.10 Incidente 10: Limpieza y Reconstrucción Limpia de la Cache en Apple Silicon

```
killall -9 Unity 2>/dev/null  
rm -rf Library/ Temp/ Obj/
```

¿Qué hace este comando?

Elimina la cache local corrupta. Al abrir el proyecto en Unity para Apple Silicon, el motor reconstruirá automáticamente todos los assets de forma limpia.